Companion to Bird & de Moor, Algebra of Programming

§1. Programs

§1.1. Datatypes

§1.2. Natural numbers

§1.3. Lists

§1.4. Trees

§1.5. Inverses

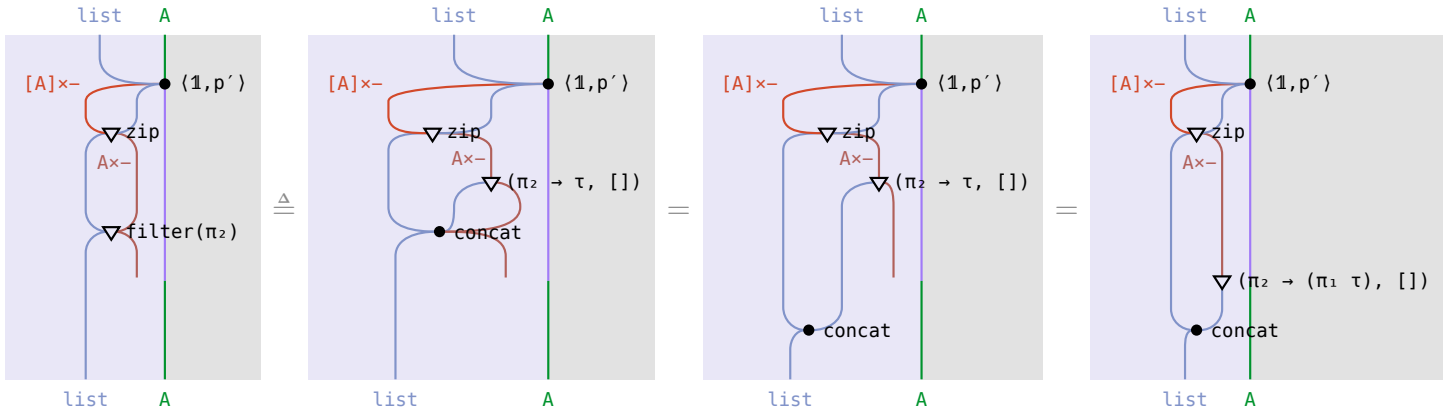
§1.6. Polymorphic functions

§1.7. Pointwise and point-free

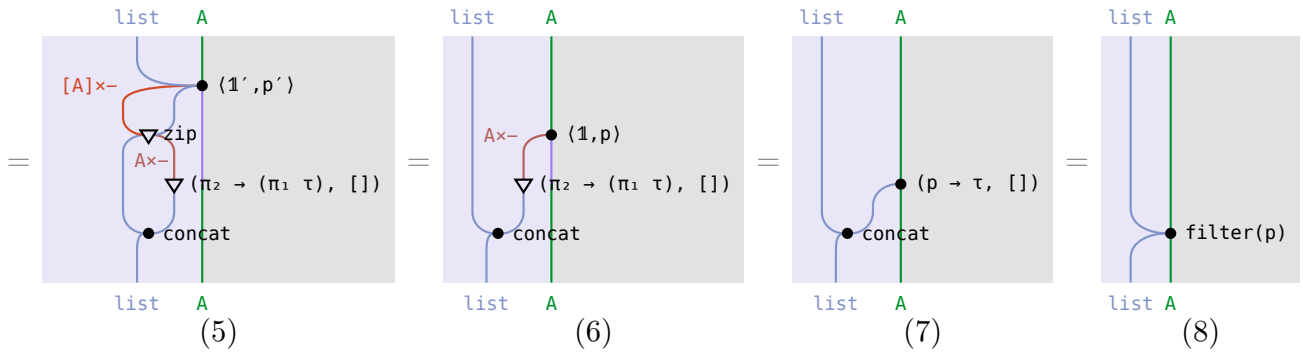
⟨𝟙,p′⟩ zip filter(π₂) π₁′ = filter(p) (1.7a)

pairing each element with its answer under **p** (**⟨𝟙,p′⟩ zip**), keeping the pairs answered **true** (**filter(π₂)**) and dropping the answers (**π₁′**) is **filter(p)**

the starting composite



1	⟨𝟙,p′⟩ zip filter(π₂) π₁′	definition of filter: filter(p) ≜ (p → τ, nil)′ concat
2	≜ ⟨𝟙,p′⟩ zip (π₂ → τ, [])′ concat π₁′	(1.6): concat f′ = f′′ concat (1.8) backwards: f′ g′ = (fg)′
3	= ⟨𝟙,p′⟩ zip ((π₂ → τ, []) π₁′)′ concat	(1.11): (p → f, g)h = (p → fh, gh); (1.5): τ f′ = f τ; (1.4): nil f′ = nil
4	= ⟨𝟙,p′⟩ zip (π₂ → (π₁ τ), [])′ concat	(1.9) backwards: 𝟙 = 𝟙′



5	= ⟨𝟙′,p′⟩ zip (π₂ → (π₁ τ), [])′ concat	(1.7): ⟨f′,g′⟩ zip = ⟨f,g⟩′ (1.8) backwards: f′ g′ = (fg)′
6	= (⟨𝟙,p⟩ (π₂ → (π₁ τ), []))′ concat	(1.10): h(p → f, g) = (hp → hf, hg); (1.1): ⟨f,g⟩π₁ = f; (1.3): f nil = nil (1.12): 𝟙 f = f
7	= (p → τ, [])′ concat	definition of filter
8	= filter(p)	